Object Oriented Programming (CSEG1044)

Unit 05 – Lecture 01: Collections Framework Overview + Iteration + List

Tofik Ali

School of Computer Science, UPES Dehradun

February 27, 2026

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Quick Links

Overview Key Topics Demo Summary

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Agenda

1 Overview

2 Key Topics

3 Demo / Example

4 Summary

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Learning Outcomes

- Explain `Iterable`, `Collection`, and `Iterator`.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Learning Outcomes

- Explain `Iterable`, `Collection`, and `Iterator`.
- Choose `ArrayList` vs `LinkedList` for common use cases.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Learning Outcomes

- Explain `Iterable`, `Collection`, and `Iterator`.
- Choose `ArrayList` vs `LinkedList` for common use cases.
- Sort a list using `Comparable` and `Comparator` (with lambdas).

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Unit 05: Collections and Wrapper Class

- Lecture focus: Collections Framework Overview + Iteration + List

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Today: Roadmap

- Core interfaces: ‘Collection‘, ‘Iterable‘, ‘Iterator‘

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Today: Roadmap

- Core interfaces: ‘Collection‘, ‘Iterable‘, ‘Iterator‘
- List interface; ‘ArrayList‘ vs ‘LinkedList‘ (when to use which)

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Today: Roadmap

- Core interfaces: ‘Collection‘, ‘Iterable‘, ‘Iterator‘
- List interface; ‘ArrayList‘ vs ‘LinkedList‘ (when to use which)
- Sorting with ‘Comparable‘/‘Comparator‘ (use lambdas where helpful)

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Collections Framework (why)

- Standard data structures + algorithms (reduce bugs).

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Collections Framework (why)

- Standard data structures + algorithms (reduce bugs).
- Works well with generics: `List<Student>`.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Collections Framework (why)

- Standard data structures + algorithms (reduce bugs).
- Works well with generics: `List<Student>`.
- Includes sorting utilities and iterators.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Iteration (safe removal)

- for-each loop uses an `Iterator` internally.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Iteration (safe removal)

- for-each loop uses an `Iterator` internally.
- Removing inside for-each can throw `ConcurrentModificationException`.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Iteration (safe removal)

- for-each loop uses an `Iterator` internally.
- Removing inside for-each can throw `ConcurrentModificationException`.
- Use `Iterator.remove()` when you need to delete while looping.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

ArrayList vs LinkedList (quick)

- `ArrayList`: fast random access; best default choice.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

ArrayList vs LinkedList (quick)

- `ArrayList`: fast random access; best default choice.
- `LinkedList`: slower random access; useful as queue/deque.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Sorting: Comparable vs Comparator

- `Comparable`: natural ordering (`compareTo`) in the class.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Sorting: Comparable vs Comparator

- `Comparable`: natural ordering (`compareTo`) in the class.
- `Comparator`: custom ordering (`compare`) outside the class.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Sorting: Comparable vs Comparator

- `Comparable`: natural ordering (`compareTo`) in the class.
- `Comparator`: custom ordering (`compare`) outside the class.
- Lambdas make comparators short and readable.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Example: List + iterator + sorting (1/2)

```
import java.util.*;

class Student {
    final String rollNo;
    final double cgpa;
    Student(String rollNo, double cgpa) { this.rollNo =
        rollNo; this.cgpa = cgpa; }
    double getCgpa() { return cgpa; }
    public String toString() { return rollNo + ":" + cgpa;
         }
}

public class Demo {
    public static void main(String[] args) {
        List<Student> list = new ArrayList<>();
        list.add(new Student("21CS1002", 9.1));
```

Example: List + iterator + sorting (2/2)

```
// continued...
        Iterator<Student> it = list.iterator();
        while (it.hasNext()) {
            if (it.next().getCgpa() < 4.0) it.remove();
        }

        list.sort((a, b) -> Double.compare(b.getCgpa(), a.
            getCgpa())); // desc
        System.out.println(list);
    }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Mini Exercises (2–3 mins)

1. Why use `Iterator.remove()` when deleting during traversal?

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Mini Exercises (2–3 mins)

1. Why use `Iterator.remove()` when deleting during traversal?
2. Which list is best for random access: `ArrayList` or `LinkedList`?

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Mini Exercises (2–3 mins)

1. Why use `Iterator.remove()` when deleting during traversal?
2. Which list is best for random access: `ArrayList` or `LinkedList`?
3. Difference between `Comparable` and `Comparator`?

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Key Takeaways

- Lists store ordered data and allow duplicates.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Key Takeaways

- Lists store ordered data and allow duplicates.
- Iterators enable safe traversal and removal.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Key Takeaways

- Lists store ordered data and allow duplicates.
- Iterators enable safe traversal and removal.
- Sorting is done using `Comparable` or `Comparator`.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Checkpoint Questions

1. What is the purpose of `Iterator`?

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Checkpoint Questions

1. What is the purpose of `Iterator`?
2. When do you choose `LinkedList` over `ArrayList`?

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Checkpoint Questions

1. What is the purpose of `Iterator`?
2. When do you choose `LinkedList` over `ArrayList`?
3. How do you sort a list by a custom rule?

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).

Next Lecture Preview

Next: Set/SortedSet + Map/SortedMap

- Uniqueness (Set) and key-value storage (Map) with hashing basics.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed February 27, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed February 27, 2026).